

## Linguagem C

**for, while, do-while, rand**

Prof. Nilton

[nilton@comp.uems.br](mailto:nilton@comp.uems.br)

# for

- O comando **for** permite que um certo trecho de programa seja executado **um determinado número de vezes**.

```
for(inicialização; expressão; incremento/decremento) {
```

```
    // comandos a serem repetidos
```

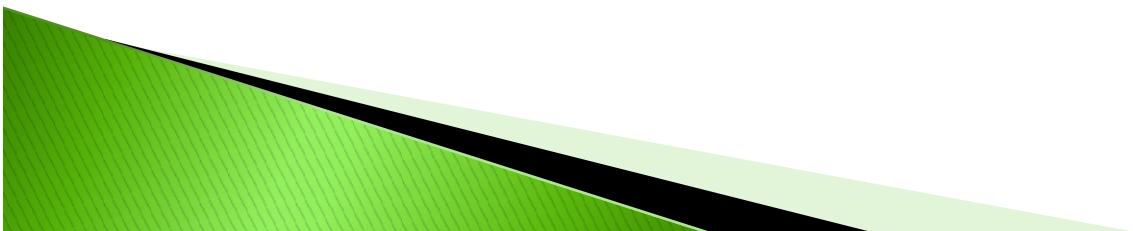
```
}
```

```
    // comandos após o for
```

1. Inicializa a variável de controle
  2. Testa a expressão
  3. Se a expressão for **falsa** então executa os comandos após o **for**
  4. Se a expressão for **verdadeira** então executa os comandos subordinados ao **for**
  5. Atualiza a variável de controle
  6. Volta ao passo 2
- 

# for

- ▶ O comando **for** deve ser usado quando:
  1. Sabe-se exatamente quantas vezes o laço deve ser repetido;
  2. O teste deve ser feito antes da execução do laço;
  3. Há casos em que o laço não seja executado.



# for

- Usando **incremento**

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int n, i;  
    float soma, num;
```

```
    printf("\nEntre com a quantidade de numeros a serem somados: ");
```

```
    scanf("%d",&n);
```

```
    soma=0;
```

```
    for(i=0; i<n; i++) {
```

```
        printf("\nDigite o %do. numero ",i+1);
```

```
        scanf("%f",&num);
```

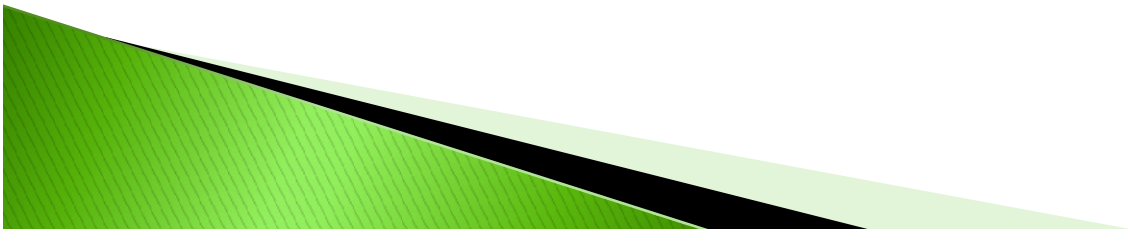
```
        soma=soma+num;
```

```
    }
```

```
    printf("\nO resultado da soma vale %f\n",soma);
```

```
    return 0;
```

```
}
```



# for

- ▶ Usando **decremento**

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int n, i;  
    float soma, num;
```

```
    printf("\nEntre com a quantidade de numeros a serem somados: ");
```

```
    scanf("%d",&n);
```

```
    soma=0;
```

```
    for(i=n; i>0; i--) {
```

```
        printf("\nDigite o %do. numero ",n-i+1);
```

```
        scanf("%f",&num);
```

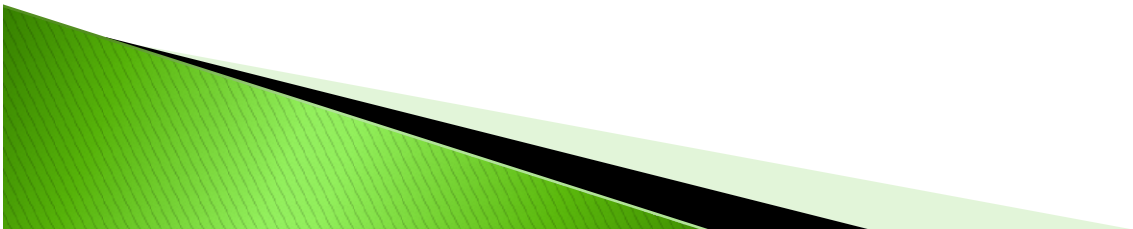
```
        soma=soma+num;
```

```
    }
```

```
    printf("\nO resultado da soma vale %f\n",soma);
```

```
    return 0;
```

```
}
```



# while

- ▶ O comando **while** permite que um certo trecho de programa seja executado **enquanto** uma certa expressão for **verdadeira**.

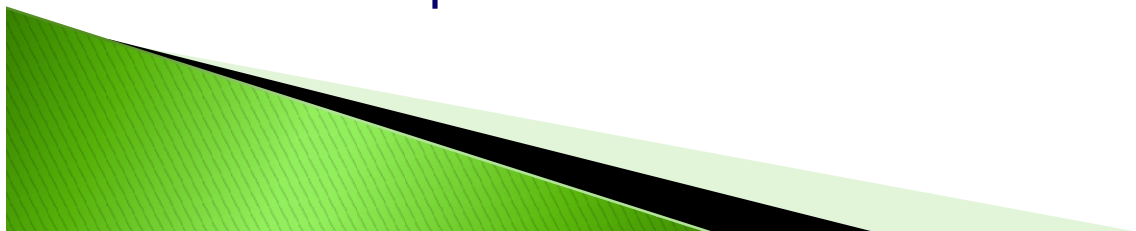
```
while(expressão) {
```

```
    // comandos a serem repetidos
```

```
}
```

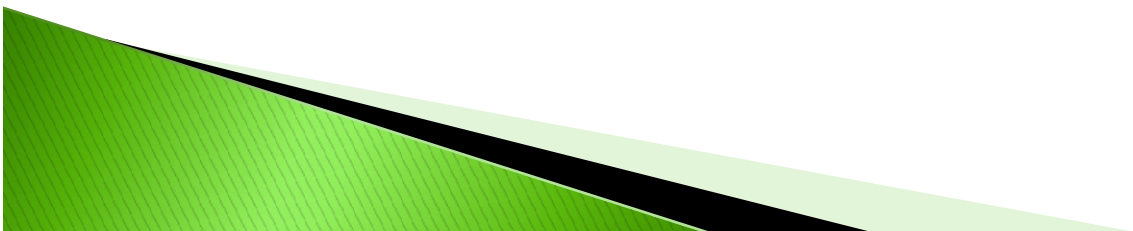
```
// comandos após o while
```

1. Testa a expressão
2. Se a expressão for **falsa** então executa os comandos após o **while**
3. Se a expressão for **verdadeira** então executa os comandos subordinados ao **while**
4. Volta ao passo 1



# while

- ▶ O comando **while** deve ser usando quando:
  1. Não se sabe exatamente quantas vezes o laço deve ser repetido;
  2. O teste deve ser feito antes da execução do laço;
  3. Há casos em que o laço não seja executado.



# while

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    float soma, num;
```

```
    soma=0;
```

```
    i=0;
```

```
    num=0;
```

```
    while(num != -1) {
```

```
        i=i+1;
```

```
        printf("\nDigite o %do. numero ",i);
```

```
        scanf("%f",&num);
```

```
        if(num != -1)
```

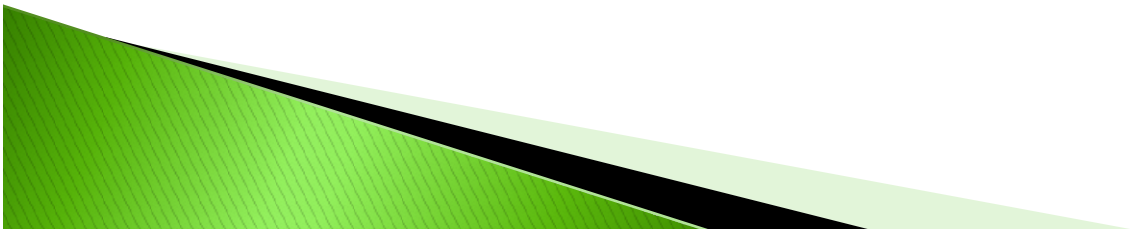
```
            soma=soma+num;
```

```
    }
```

```
    printf("\nO resultado da soma vale %f\n",soma);
```

```
    return 0;
```

```
}
```





# do-while

- ▶ O comando **do-while** permite que um certo trecho de programa seja executado enquanto uma certa expressão for verdadeira.

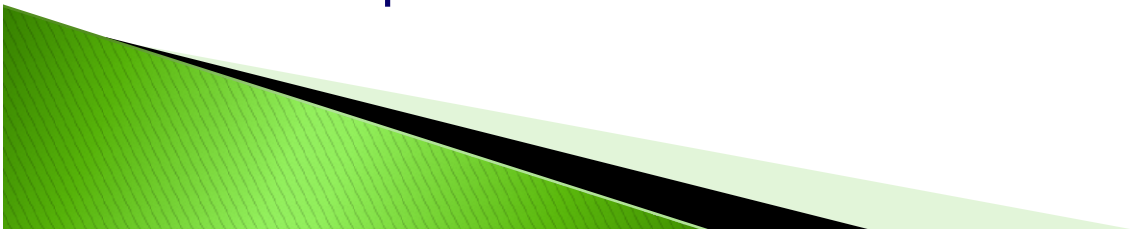
```
do {
```

```
    // comandos a serem repetidos
```

```
} while(expressão);
```

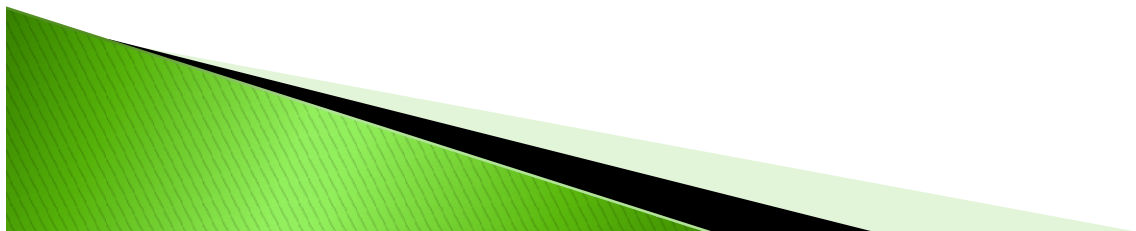
```
// comandos após o do-while
```

1. Executa os comandos subordinados ao **do-while**
2. Testa a expressão
3. Se a expressão for **falsa** então executa os comandos após o **do-while**
4. Se a expressão for **verdadeira** então volta ao passo 1



# do-while

- ▶ O comando **do-while** deve ser usando quando:
  1. Não se sabe exatamente quantas vezes o laço deve ser repetido;
  2. O teste deve ser feito depois da execução do laço;
  3. O laço deve ser executado pelo menos uma vez.



# do-while

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    float soma, num;
```

```
    soma=0;
```

```
    i=0;
```

```
    do {
```

```
        i=i+1;
```

```
        printf("\nDigite o %do. numero ",i);
```

```
        scanf("%f",&num);
```

```
        if(num != -1)
```

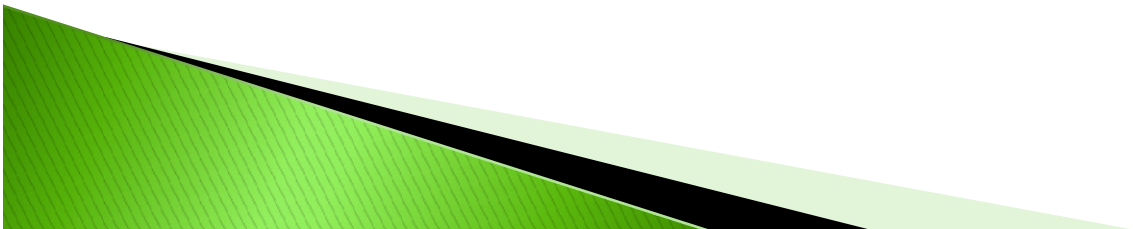
```
            soma=soma+num;
```

```
    } while(num != -1);
```

```
    printf("\nO resultado da soma vale %f\n",soma);
```

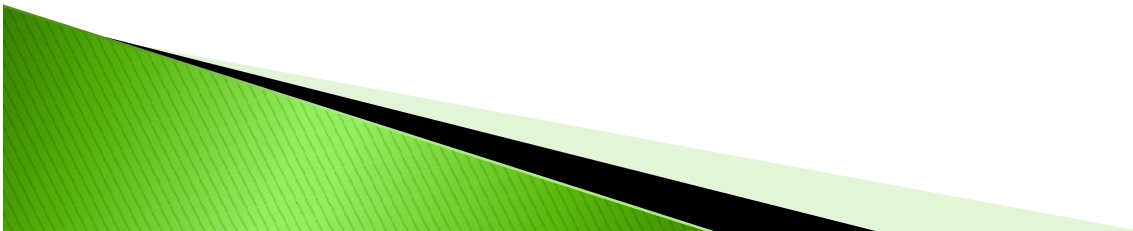
```
    return 0;
```

```
}
```



# Gerando número aleatórios

- ▶ Para gerar um número aleatório (randômico) em linguagem C pode-se utilizar a função *rand* pertencente à biblioteca `stdlib.h`;
- ▶ A função `rand` produz um valor aleatório na faixa entre 0 e a constante `RAND_MAX`. O valor dessa constante encontra-se no arquivo `stdlib.h`, podendo ser 32.767;
- ▶ Para gerar um valor aleatório dentro de uma determinada faixa de valores (diferente entre 0 e `RAND_MAX`) usa-se o cálculo de módulo (%).
- ▶ Por exemplo, para gerar valores entre 0 e 999 aplica-se a expressão `rand() % 1000`;



# Ex. Gerando valores entre 1 e 1000

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
```

```
int main(void) {
    int i;

    printf("Gerando 10 valores aleatorios entre 1 e 1000:\n\n");
    for (i = 0; i < 10; i++) {
        printf("%d ", (rand() % 1000)+1);
    }
    getch();
    return 0;
}
```

- ▶ Ao executar o programa os seguintes números são gerados {42, 468, 335, 501, 170, 725, 479, 359, 963, 465}
- ▶ Executando o programa várias vezes o que ocorre?



# Ex. Gerando valores entre 50 e 100

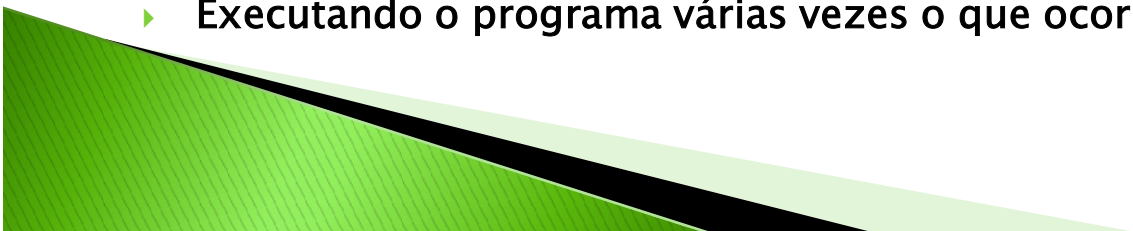
```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
```

```
int main(void) {
    int n, cont=0;

    printf("Gerando 10 valores aleatorios entre 50 e 100:\n\n");
    do {

        n = (rand() % 100) + 1;
        if((n >= 50) && (n <= 100)) {
            printf("%d ", n);
            cont++;
        }
    } while(cont < 10);
    getch();
    return 0;
}
```

- ▶ Ao executar o programa os seguintes números são gerados  
{68, 70, 79, 59, 63, 65, 82, 62, 92, 96}
- ▶ Executando o programa várias vezes o que ocorre?



# Ex. Gerando valores entre 30 e 50

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
```

```
int main(void) {
    int n, cont=0;

    printf("Gerando 10 valores aleatorios entre 30 e 50:\n\n");
    do {
        do {
            n = rand();
        } while(n<30 || n>50);
        printf("%d ", n);
        cont++;
    } while(cont<10);
    getch();
    return 0;
}
```

- ▶ Ao executar o programa os seguintes números são gerados {41, 40, 42, 39, 35, 31, 47, 45, 37, 42}
- ▶ Executando o programa várias vezes o que ocorre?



## Ex. Gerando valores entre 30 e 50 ou 100 e 150 ou 400 e 450

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
```

```
int main(void) {
    int n, cont=0;

    printf("Gerando 10 valores aleatorios entre 30 e 50 ou 100 a 150 ou 400 a 450:\n\n");
    do {
        do {
            n = rand();
        } while( ? );
        printf("%d ", n);
        cont++;
    } while(cont<10);
    getch();
    return 0;
}
```

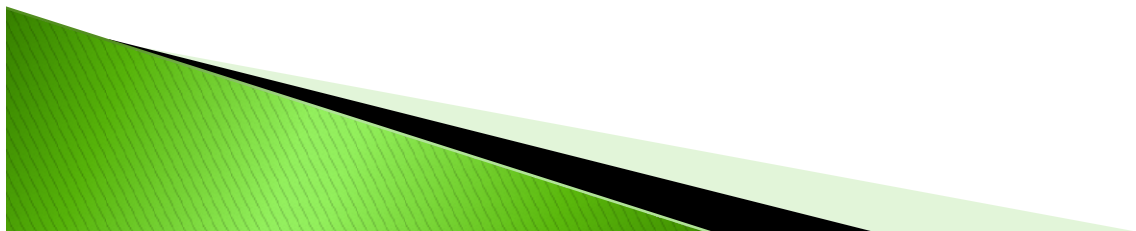
- ▶ Ao executar o programa os seguintes números são gerados  
{41, 142, 140, 123, 148, 406, 144, 40, 119, 113}
- ▶ Executando o programa várias vezes o que ocorre?





# Mais sobre a função rand()

- ▶ Ocorre que para a função *rand* funcionar ela precisa de um valor inicial chamado de “semente”. Se nenhum valor é passado para *rand* é assumido o valor 1 e por isto a mesma seqüência sempre é gerada em execuções diferentes do programa;
- ▶ Para gerar valores diferentes a cada execução é necessário utilizar a função *srand* que inicializa a função *rand* com um valor “semente” de tal forma que esta semente seja um valor diferente a cada execução do programa, isto por sua vez produz valores diferentes na seqüência;



# Ex. Gerando valores entre 1 e 1000

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>
```

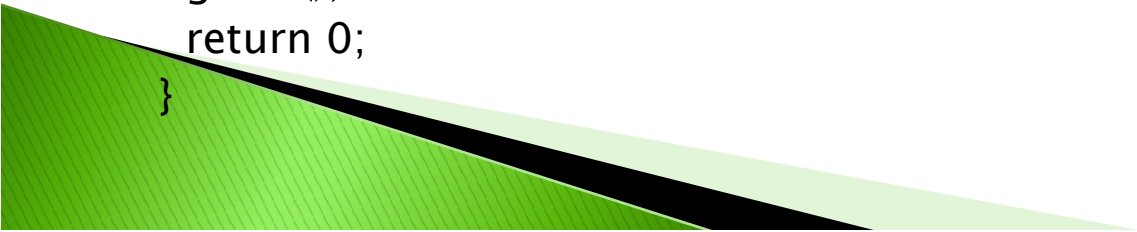
```
int main(void) {
    int i;
```

```
    // o valor obtido de time(NULL) como semente para o rand garante a
    // geração de números aleatórios diferentes em cada execução do programa.
    // Neste caso time(NULL) devolve um valor em segundos desde 01/01/1970
    // até a data atual;
```

```
    srand(time(NULL));
```

```
    printf("Gerando 10 valores aleatorios entre 1 e 1000:\n\n");
```

```
    for (i = 0; i < 10; i++) {
        printf("%d ", (rand() % 1000)+1);
    }
    getch();
    return 0;
}
```



# Exercícios

- ▶ Gerar 5 números aleatórios entre -50 e 50.
- ▶ Implementar os algoritmos da listas 3 e 3.1 usando a linguagem C
- ▶ Estudar o **comando switch** para substituir alguns casos de encadeamento if-else-if

